

— WEB DEVELOPMENT MODULE

CSS Capstone: Style a Complete Programme Page

The final mission: take the raw HTML structure we built and transform it into a professional, responsive LEA Labs landing page.

MODULE

CSS & Layout

FORMAT

Project Walkthrough

GOAL

A Production-Ready Page

Establishing the Design System

Before styling any elements, we define our design tokens—the single source of truth for colors, fonts, and spacing.

THE ROOT OF CONSISTENCY

By declaring custom properties (variables) on the `:root` pseudo-class, we make our design tokens available globally. If the brand color changes later, we only update it in one place.

SEMANTIC NAMING

Name variables based on their *purpose*, not their literal value. Use `--color-brand` instead of `--purple`, so the name remains accurate even if the color shifts.

```
:root {  
  /* Colors */  
  --color-brand: #4d176e;  
  --color-accent: #f47945;  
  --color-dark: #1f0d2e;  
  --color-light: #fffdfb;  
  
  /* Typography */  
  --font-main: 'Outfit', sans-serif;  
}
```

`--color-brand`

`--color-accent`

`--color-dark`

`--color-light`

The CSS Reset

Browsers apply default styles. A reset evens things out so our design looks consistent across platforms.

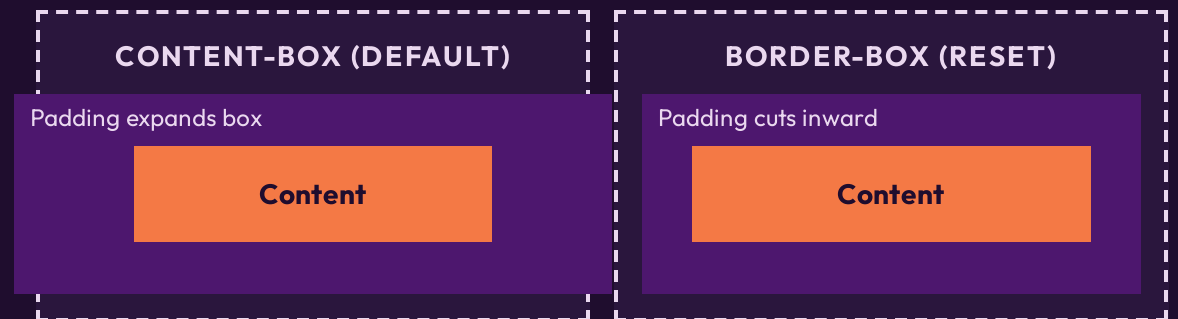
FIXING THE BOX MODEL

Padding and borders normally enlarge an element. Using `box-sizing: border-box` keeps sizing predictable by keeping padding inside the box.

REMOVING DEFAULTS

We remove default margins from headings and paragraphs so spacing is intentional and consistent.

```
/* The universal reset */  
*, *::before, *::after {  
  box-sizing: border-box;  
  margin: 0;  
  padding: 0;  
}
```



Base Typography

We apply the font family to the body and set up fluid scaling so text stays readable across sizes.

THE BODY FOUNDATION

Set the font family and base color on body so text inherits consistent typography and spacing.

FLUID HEADINGS

Use `clamp()` for headings so they scale between a min and max size without breakpoint rules.

```
body { font-family: var(--font-main); line-height: 1.6; }  
h1 { font-size: clamp(2.5rem, 5vw, 4rem); line-height: 1.1; }
```

Heading 1 (Fluid)

Heading 2

Body text uses Outfit with a generous line-height for readability.

The Layout Shell

We use CSS Grid on the body to ensure the footer always stays at the bottom of the screen, even if content is sparse.

FULL HEIGHT VIEWPORT

Setting `min-height: 100vh` ensures the body takes up at least the full height of the browser window.

GRID TEMPLATE ROWS

By defining `grid-template-rows: auto 1fr auto`, the header and footer size to their content (`auto`), while the main content area stretches to fill all remaining space (`1fr`).

```
body {  
  display: grid;  
  grid-template-rows: auto 1fr auto;  
  min-height: 100vh;  
}
```

HEADER (AUTO)

MAIN CONTENT (1FR)

FOOTER (AUTO)

Styling the Header

Flexbox is ideal for nav bars, letting us space the logo and links on one row.

[PROGRAMMES](#)[ABOUT](#)[LOGIN](#)

```
.site-header {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
}  
  
.nav-list {  
  display: flex;  
  gap: var(--spacing-lg);  
}
```

THE FLEX CONTAINER

Making the header a flex container places logo and nav on one row.

SPACE BETWEEN

`justify-content: space-between` pushes the logo and nav to opposite sides.

VERTICAL ALIGNMENT

`align-items: center` vertically centers items in the header.

The Hero Section

The hero section is the first thing users see. We use bold colors, centered alignment, and a clear call-to-action to grab attention.

IMPACTFUL SPACING

Generous padding gives the hero section breathing room. We use `text-align: center` to focus the user's eye directly on the core message.

THE CALL-TO-ACTION (CTA)

Buttons need to stand out. We apply our accent color (`--color-accent`), bold typography, and uppercase text to make the primary action unmistakable.

```
.hero {  
  background-color: var(--  
color-dark);  
  padding: 80px 20px;  
  text-align: center;  
}  
  
.btn-primary {  
  background-color: var(--  
color-accent);  
  padding: 16px 32px;  
}
```

**Build the skills
your next role
will ask for.**

LEA Labs is a practical
learning space for ambitious
people who want to move
with confidence.

**EXPLORE
PROGRAMMES**

The Outcomes Cards

We use CSS Grid's auto-fit feature to create a responsive card layout that adapts to any screen size without media queries.

THE MAGIC OF AUTO-FIT

`repeat(auto-fit, minmax(300px, 1fr))` tells the browser to fit as many 300px columns as possible into the container. If there isn't enough room, the cards wrap to the next line automatically.

CARD STYLING

The cards themselves use Flexbox to stack their internal content (icon, title, text) vertically with a consistent gap, while maintaining a uniform height across the row.

```
.outcomes-grid {  
  display: grid;  
  grid-template-columns:  
    repeat(auto-fit,  
    minmax(300px, 1fr));  
  gap: 32px;  
}  
  
.card {  
  display: flex;  
  flex-direction: column;  
  gap: 16px;  
}
```



Build Real Projects

Learn by doing with hands-on labs.



Modern Stack

Master HTML, CSS, and JavaScript.

The Schedule Table

Tables can be hard to read without proper styling. We use borders, padding, and alternating row colors to make data scannable.

BORDER COLLAPSE

Table cells by default have separate borders. `border-collapse: collapse` merges them into single, clean lines for a modern look.

ALTERNATING ROWS (ZEBRA STRIPING)

Using `:nth-child(even)`, apply a subtle background to every other row to help the eye track across data.

```
table { width: 100%; border-collapse: collapse; }  
th, td { padding: 8px 12px; border-bottom: 1px solid  
var(--color-light); }  
tr:nth-child(even) { background-color: var(--color-light-  
alt); }
```

WEEK	TOPIC	FORMAT
1 - 4	HTML & CSS Foundations	Online
5 - 8	JavaScript & DOM	Hybrid
9 - 12	React & State	Online

The Registration Form

Use clear labels and states to make forms easy to scan and use.

INPUTS AND LABELS

Labels sit above inputs and inputs get ample padding for easy clicking.

FOCUS STATES

Focus changes border and adds an outline so keyboard users can follow along.

THE SUBMIT BUTTON

The primary action uses a high-contrast accent and clear wording.

```
input, select { padding: 12px; border: 2px solid var(--color-light); }  
input:focus { border-color: var(--color-accent); outline: 2px solid var(--color-accent); }
```

Full Name

Jane Doe

Email Address

jane@example.com

REGISTER NOW

Responsive Polish

Even with fluid typography and auto-fit grids, some elements need specific adjustments on smaller screens using media queries.

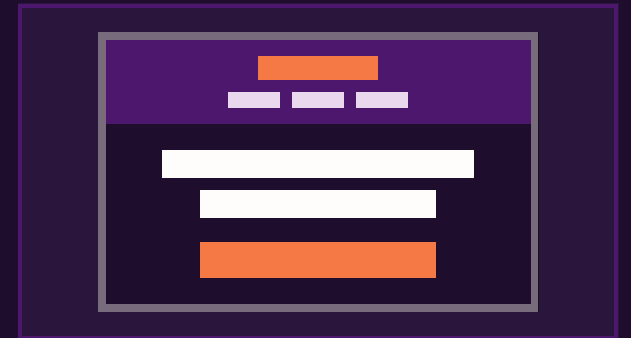
STACKING THE HEADER

On mobile devices, a horizontal navigation bar can become cramped. We use a media query to change the header's flex direction to `column`, stacking the logo above the links.

REDUCING SPACING

Generous padding looks great on desktop, but wastes valuable screen real estate on mobile. We reduce the hero section's padding to maximize visible content.

```
@media (max-width: 768px) {  
  .site-header {  
    flex-direction: column;  
    gap: 16px;  
  }  
  
  .hero {  
    padding: 40px 16px;  
  }  
}
```



Lab: Style the Programme Page

Take the raw HTML structure from the previous module and apply the complete CSS design system to build a production-ready landing page.

THE BRIEF

You are provided with the unstyled HTML for the LEA Labs Web Development programme page. Your task is to write the CSS to match the provided design mockup and ensure responsiveness.

KEY FOCUS AREAS

- Design tokens & CSS reset
- Flexbox header & Grid layout
- Auto-fit cards & forms

DELIVERABLES

- A global stylesheet defining custom properties and typography.
- Component styles for header, hero, cards, table, and form.
- Media queries that adjust the layout for mobile and tablet.

SUCCESS CRITERIA

- The page visually matches the LEA Labs design specs.
- No horizontal scrolling occurs at any viewport width.